arXiv:2405.00263v1 [cs.CL] 1 May 2024

Clover: Regressive Lightweight Speculative Decoding with Sequential Knowledge

Bin Xiao[1] **Chunan Shi**[2] **Xiaonan Nie**[1 2] **Fan Yang**[1] **Xiangwei Deng**[2]
Lei Su[1] **Weipeng Chen**[1] **Bin Cui**[2]
[1]Baichuan Inc. [2] Peking University
{xiaobin, yangfan, sulei, chenweipeng}@baichuan-inc.com
{spirited_away, xiaonan.nie, bin.cui}@pku.edu.cn, dengxiangwei@stu.pku.edu.cn

Abstract

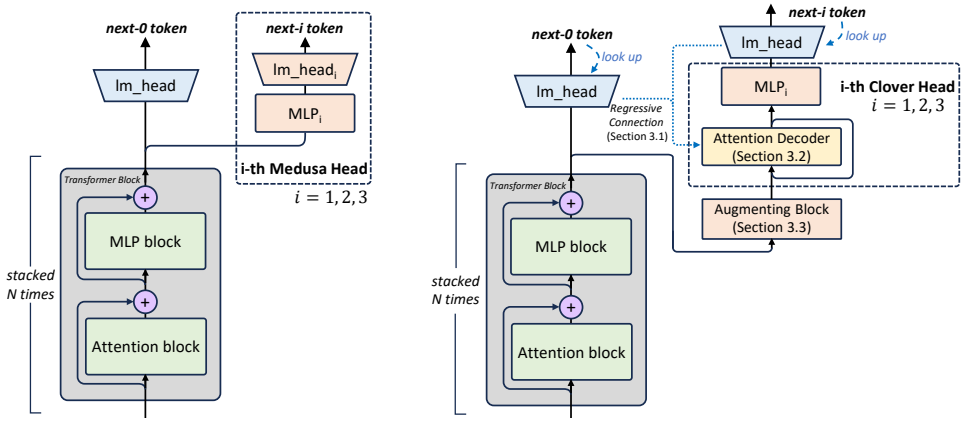
Large language models (LLMs) suffer from low efficiency as the mismatch between the requirement of auto-regressive decoding and the design of most contemporary GPUs. Specifically, billions to trillions of parameters must be loaded to the GPU cache through its limited memory bandwidth for computation, but only a small batch of tokens is actually computed. Consequently, the GPU spends most of its time on memory transfer instead of computation. Recently, parallel decoding, a type of speculative decoding algorithms, is becoming more popular and has demonstrated impressive efficiency improvement in generation. It introduces extra decoding heads to large models, enabling them to predict multiple subsequent tokens simultaneously and verify these candidate continuations in a single decoding step. However, this approach deviates from the training objective of next token prediction used during pre-training, resulting in a low hit rate for candidate tokens.

In this paper, we propose a new speculative decoding algorithm, `Clover`, which integrates sequential knowledge into the parallel decoding process. This enhancement improves the hit rate of speculators and thus boosts the overall efficiency. `Clover` transmits the sequential knowledge from pre-speculated tokens via the `Regressive Connection`, then employs an `Attention Decoder` to integrate these speculated tokens. Additionally, `Clover` incorporates an `Augmenting Block` that modifies the hidden states to better align with the purpose of speculative generation rather than next token prediction. We conducted experiments on both Baichuan-Small (with 7B parameters) and Baichuan-Large (with over 100B parameters). The results demonstrate that `Clover` achieves superior performance compared to existing methods across different model sizes. Specifically, `Clover` outperforms the baseline by up to 91% on Baichuan-Small and 146% on Baichuan-Large, respectively, and exceeds the performance of the previously top-performing method, Medusa, by up to 37% on Baichuan-Small and 57% on Baichuan-Large, respectively.

1 Introduction

Generative large language models (LLMs) [18, 1, 4], such as GPT, represent a significant breakthrough in artificial intelligence. They have demonstrated remarkable proficiency across a diverse range of applications, from composing creative literary works to generating human-like dialogues in chatbots. Their capability to understand and produce language has opened up new avenues for human-computer interaction, automating tasks that necessitate an understanding of context and nuance.

However, despite their strong capabilities, LLMs also present significant challenges related to low generation efficiency on GPUs. Specifically, these models create text sequentially by generating one output token per step, responding to a user query in two distinct phases: the prefilling phase and the



(a) Medusa Decoding (b) `Clover` Decoding

Figure 1: Overview of Medusa decoding and our extended `Clover` Decoding.

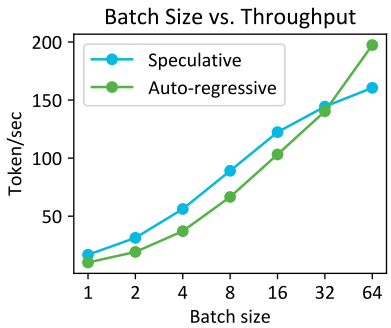
decoding phase. (1) During the prefilling phase, the model processes all tokens in the input prompt or context in a single iteration to generate the initial output token. (2) During the decoding phase, the model, informed by the prompt/context and previously generated tokens, continues to produce subsequent output tokens one at a time through multiple iterations until the response is complete. Due to the decoding phase involving multiple rounds of generation, where each round processes only a small batch of tokens, the GPUs' computational resources are severely underutilized.

Speculative decoding [13, 6] is an acceleration technique used to mitigate the performance issues in question. It increases computational density by generating multiple tokens in a single step while ensuring the outputs remain entirely consistent. Specifically, speculative decoding involves one or more lightweight draft models that speculate multiple subsequent tokens with negligible overhead. These speculations are then verified by the original target model, which generates multiple tokens in a single iteration. Speculative decoding generates multiple tokens based on initial speculations. The accuracy of these speculators is critical for decoding speed, while more complex speculators increase inference overhead, subsequently extending latency. Numerous studies [16, 15, 17, 19, 28, 27, 11] have explored enhancing latency and throughput using independent draft models as speculators. Additionally, recent discussions [5, 14, 3, 2, 26, 25, 8] have highlighted the advantages of integrated speculators, noting their lightweight nature and ease of deployment.

The Medusa solution [5] leverages lightweight heads as speculators. As shown in Figure 1a, it features multiple parallel MLP (Multi-Layer Perceptron) layers that receive inputs from the hidden states of the last transformer block. Each layer is designed to predict a single subsequent token and utilizes a tree-based verification process to simultaneously generate multiple tokens and initiate new speculations. This lightweight head mechanism has led to substantial improvements in inference speed.

However, Medusa still encounters several challenges that can hinder its performance. Firstly, the Medusa head consists of only a single MLP layer that takes input solely from the final hidden states. Each layer independently speculates on a word at a specified position beyond the next, disregarding the sequential dependencies from previously predicted tokens, which often results in decreased accuracy. Secondly, because the Medusa heads operate independently, the tokens they speculate are combined using a Cartesian product to form an exponentially large token tree. This approach can lead to suboptimal performance when the decoding phase is not constrained by memory, as it generates a surplus of redundant tokens, particularly as the batch size increases. Additionally, the absence of sequential information compromises the effectiveness of the tree pruning algorithm, further impacting performance.

In real-time serving scenarios, where the inference batch size is typically large, speculative decoding often faces computational constraints, leading to performance degradation. Figure 2 illustrates this trend: speculative decoding substantially outperforms auto-regressive decoding when the number of computed tokens is low. However, as the token count increases, the speedup provided by speculative



t

Figure 2: Throughput on a model with approximately 30B parameters, supposing speculation length is 5 with 0.4 acceptance rate.

decoding reaches an inflection point and gradually diminishes due to computational limitations. Consequently, the actual size of the token tree in practice is usually smaller than what is assumed in previous studies.

To address these issues, we introduce `Clover`, an enhancement of the Medusa framework. `Clover` incorporates a regressive attention block into the speculative phase and introduces the `Regressive Connection` (Section 3.1), `Attention Decoder` (Section 3.2), and `Augmenting Block` (Section 3.3). These components enable speculators to utilize additional sequential knowledge, enhancing their accuracy. Moreover, the regressive architecture not only improves the precision of the speculations but also generates a token tree with more comprehensive dependency information.

We evaluate `Clover` in a setting that more closely resembles the real scenario, which involve various larger batch sizes and a smaller token tree. The results on Baichuan model family show that `Clover` method achieves a maximum throughput improvement of $2.56\times$ throughput improvement over vanilla decoding and $1.25\times$ - $1.43\times$ over Medusa decoding. Moreover, `Clover` demonstrates an 11.7% - 26.4% improvement in accuracy on speculative heads, with a particularly notable increase of over 20% in the latter heads. Additionally, it generates 50% - 76% more extra tokens (except the first) per step than the Medusa method, thanks to the regressive mechanism.

To summarize, our contributions can be outlined as follows:

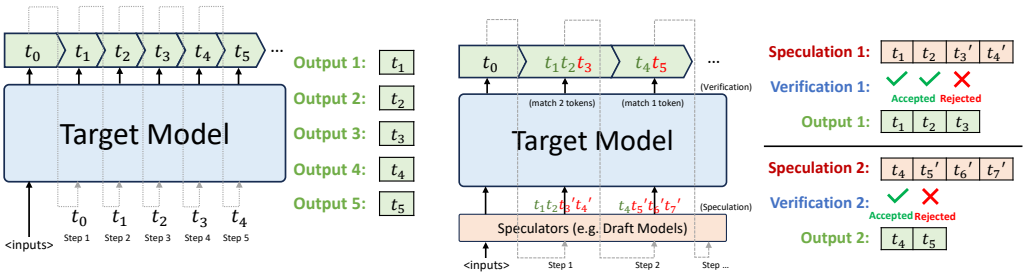
- We propose `Clover`, a new speculative decoding algorithm which incorporates an additional auto-regressive attention block to facilitate the consideration of sequential knowledge.
- We introduce three key components to improve the original parallel decoding algorithms, including the `Regressive Connection` for utilizing sequential information from previously speculated tokens, the `Attention Decoder` for combining the speculated tokens with current inputs, and the `Augmenting Block` for modifying the hidden states to better align with the purpose of speculative generation.
- Evaluations are conducted on both Baichuan-Small (with 7B parameters) and Baichuan-Large (with over 100B parameters). And results show that our `Clover` achieved better efficiency compared to existing methods, such as Medusa.

2 Background

2.1 Speculative Decoding

Speculative decoding [13, 6], depicted in Figure 3b, is an advanced technique that accelerates LLM inference by leveraging hardware computational resources more efficiently. This method distinguishes itself from traditional auto-regressive decoding by calculating and generating multiple tokens simultaneously in each iteration.

At the core of speculative decoding lies a speculator component, usually a smaller model often referred to as the draft model, which predicts several subsequent tokens. This approach contrasts with



(a) Auto-regressive Decoding (b) Speculative Decoding (maximal speculation length is 4)

Figure 3: The comparison between Auto-regressive Decoding and Speculative Decoding. Speculative Decoding may generate multiple tokens in a single step based on the speculation, thus achieves less decoding iteration and lower inference latency.

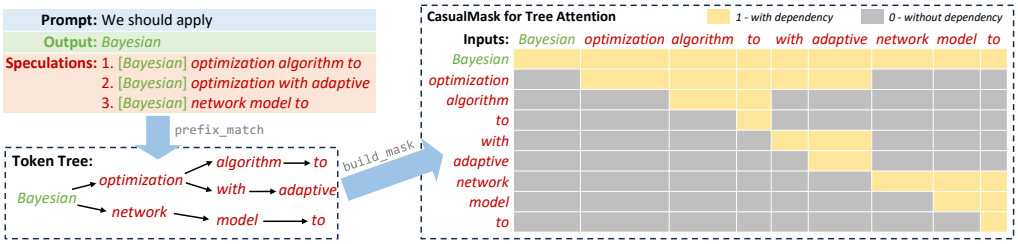


Figure 4: A demonstration of Tree Attention in Speculative Decoding. Multiple speculations are merged by prefix matching to form a tree, and its topology dependency is represented in a 2-D matrix as the casual mask in Attention computation.

auto-regressive decoding, where only the last generated token is fed into the system. In speculative decoding, the original LLM (the target model) receives all speculated tokens as input. This allows the target model to compute attention scores and derive logits over multiple tokens effectively, ensuring that it can generate consistent outputs within a single iteration. This stage is termed the *verification phase*, during which the target model screens out any incorrect tokens from the speculations. As a result, speculative inference can produce equivalent outputs with fewer decoding steps, thereby enhancing latency efficiency.

2.2 Tree Attention

Tree Attention [16] is utilized to calculate attention scores for multiple speculations in parallel. By applying prefix matching to various speculated sequences, the speculation results are organized into a *token tree*, which is represented as a 2-D matrix (Figure 4).

It is important to note that the attention block is the only component within the modern LLM architecture that requires knowledge of sequential dependency. The scoring of tree-structured tokens is a relatively straightforward task and can be achieved by configuring the attention's Causal-Mask to align with the topological matrix. Tree Attention facilitates the integration of multiple speculations with minimal computational overhead, a feature widely implemented in many speculative decoding systems such as [10, 24, 20].

2.3 Medusa Decoding

Figure 1a illustrates the Medusa architecture [5], which features several independent and parallel MLP heads. Each of these heads, designated as the i-th head, is specifically fine-tuned to predict the next-i token following the actual output token during each iteration. These lightweight heads constitute the speculator component of the Medusa system, seamlessly integrated into the target model. This integration allows for simultaneous speculation and verification within the decoding process. The design of these heads enables Medusa to effectively manage the balance between

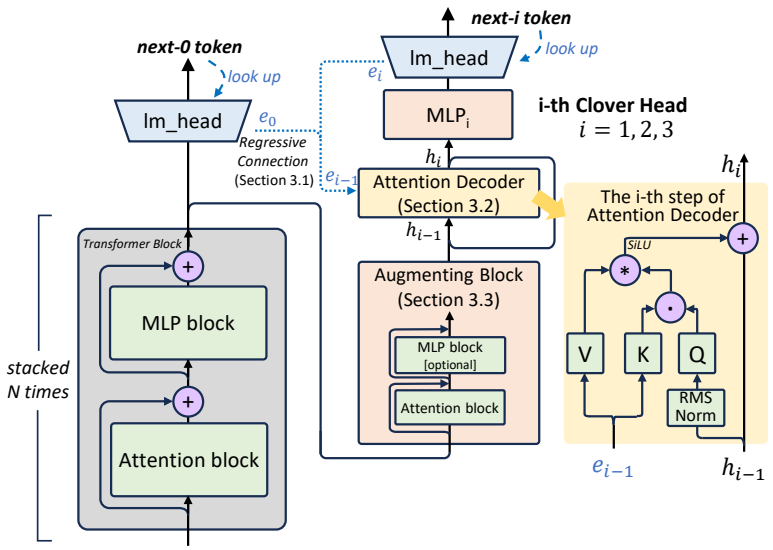


Figure 5: Detailed architecture design of `Clover`.

computational efficiency and predictive accuracy, ensuring that each token generated contributes optimally to the overall sequence coherence and context relevance.

3 Clover Design

Figure 5 shows how `Clover` is integrated into existent LLM as the speculator. `Clover` introduces three incremental components to leverage sequential knowledge: `Regressive Connection`, `Attention Decoder` and `Augmenting Block`. The Regressive Connection enables sequential dependency from preceding speculated tokens to be considered when a speculator generating the next token. The Attention Decoder is the factual regressive block in `Clover`, combining the hidden states from the last transformer block and previously speculated token, merging sequential knowledge between pre-speculated tokens and the entire input sentence. While the Augmenting Block is an additional transformer or self-attention block appended to the target model, used for enhancing sequence features to improve speculator accuracy.

3.1 Regressive Connection

Each Medusa head is responsible for speculating the token at the specified location, without considering the pre-generated speculation, as shown in Figure 1a. Although such independence enables multiple heads compute in parallel , the neglect of sequence dependencies limits the hit rate of speculation, and further increases the inference latency.

`Clover` applies regressive connection to the speculator, depicted as the blue dotted lines in Figure 5. The embedding vectors of current speculated tokens will be regressively used to predict the token at the next position. Introducing such sequential dependency knowledge offers two benefits: Firstly, speculative heads are able to generate predictions more accurately with previous tokens known, thus decrease inference latency. Although the critical path of the computation becomes proportional to the depth of the speculation and loses a certain amount of parallelism, the increase in speculation accuracy rather improves the overall latency.

Secondly, since every speculated token in the latter position has one token in its previous location as the precursor, the token tree for verification phase can have greater information density. In contrast to the exponentially sized token tree that result from the independence of words at each position, smaller token tree with sequence-dependency information is easy for pruning and less likely to meet computation bound on modern GPUs, while introducing negligible information loss.

3.2 Attention Decoder

`Clover` introduces cross attention decoder as the actual regressive block. The decoder takes two vectors as inputs: the embedding vector from the previous token, and the hidden states throughout the speculation. Specifically, considering the computation flow on the i-th head, to generate the next-i token (denoted as tok_i), the inputs of cross attention decoder are: the normalized embedding vector of the token tok_{i-1} (denoted as e_{i-1}), and the hidden states from the last speculation step (denoted a h_{i-1}). The computation can be formulated as follows:

$$Q_i = W_Q \cdot normalize(h_{i-1}),\ K_i = W_K \cdot e_{i-1},\ V_i = W_V \cdot e_{i-1}, \tag{1}$$

$$h_i = h_{i-1} + Attention(Q_i, K_i, V_i), \tag{2}$$

, where h_i is the output of the cross attention decoder, fed into the corresponding MLP layer to generate tok_i. For the first head, the hidden states h_0 comes from the last transformer block of the target LLM model (or the Augmenting Block, see below), and the embedding e_0 is from the next-0 token t_0 generated by the target model.

The hidden states h_i is recursively propagated throughout the entire speculation phase, piggybacking the features from the entire input sentence. The role of `Clover`'s Attention Decoder is combining and resolving the information from both input sentence and the previous speculated tokens, assisting the succeeding MLP layer to speculate token at present position with more sequential knowledge. We also explore the effectiveness of using the MLP layer as a regressive block, but get sub-optimal performance (more details in Section 4.3), this is probably because simply concatenating the two input vectors makes it harder to learn and extract valid features. `Clover`'s Attention Decoder has negligible overhead due to the fact that the inputs are only two vectors per request or beam.

3.3 Augmenting Block

The original target LLM is pre-trained for just predicting the next token. To extract more information for speculators to predict more succeeding tokens, we append an additional transformer block to augment features from the entire input sentence. The output of this additional augmenting block (i.e. h_0) is fed into Attention Decoder. Introducing such a whole layer incurs just a small computation overhead (e.g. approximately $1/N_{layer}$ of inference time), while the accuracy gain from the augmenting block outweighs the time it consumes.

We explore different architectures to build this augmenting block,and find the phenomenon that the attention block contributes the largest accuracy gain to all the speculative heads. We also notice that the MLP block only adds approximately 1% accuracy gain, so we leave the MLP layer in Augmenting Block optional. We still add the MLP block in our `Clover` implementation since it does indeed increase accuracy, while incurring only negligible overhead. Concerning evaluation results are discussed in Section 4.3.

3.4 Other Details

Each medusa head equipped with an individual LM head, containing a large amount of parameters (i.e. the hidden size multiplies the vocabulary size) and make it more time-consuming for training. In `Clover`, all the speculative heads share the original LM head in the target model. Furthermore, in regressive connection, the embedding vector of last generated token is given by LM head as well (the look up arrow in Figure 1b). Specifically, the embedding vector e_i is given by: the one-hot vector of token t_i multiplied by the transposed normalized weight matrix in the LM head. Compared with looking up from the embedding table, we believe such embedding distribution is much closer to the hidden states from the last transformer block, where the weights are used to initialize the augmenting block, reducing the difficulty of fine-tuning.

4 Evaluation

4.1 Experiment Settings

Models and baselines Both the Medusa and `Clover` approaches are employed on the `Baichuan Small` (with 7B parameters) and `Baichuan Large` (with over 100B parameters) models [21] with

the number of lm head is 3, named as Medusa(Baichuan) and `Clover`Baichuan, respectively. In order to ensure the fairness of the comparison, the same inference engine, tree construction and tree sampling algorithm are used for all scenarios. We also evaluate auto-regressive decoding under the same circumstances.

Dataset We employ the Baichuan internal supervised fine-tuning (SFT) dataset, containing approximately 0.15B tokens, 95% of which are Chinese, to train both Medusa(Baichuan) and `Clover` (Baichuan). We then evaluate inference performance on another internal Baichuan dataset, which consists of a variety of tasks: retrieval augmentation(**RA**), multi-turn conversation(**MC**), code(**Code**), information process(**IP**), creation(**CA**), logical reasoning(**RS**), math(**Math**), tabular(**Tab**), question answering(**QA**) and medical suggestion(**Med**). Each of the ten tasks contains 100 dialogues.

Training Both models are trained with all weights frozen in the target model . For Medusa(Baichuan), the initial weight settings correspond to the configuration given in the Medusa technical report [5]. While for `Clover` (Baichuan), the initial weights in the Augmenting Block are identical to the last transformer block in the target model, and the initialization of the MLP layer is the same as in Medusa's method. For the Attention Decoder, the weights of Q and K are initialized with identical matrix with Gaussian noise added, while the V matrix is set to all zero. We train the heads for 1 epoch, with $(\beta_1 = 0.9, \beta_2 = 0.999)$ for the AdamW optimizer. The learning rate[1] is set to 1e-3 for `Baichuan Small`, and 6e-4 for `Baichuan Large`. For both models equipped with `Clover`, the trainable parameters are approximately 0.2B and 2B, taking 2 hours to train on 8x A800 NVIDIA GPU and 32x H800 NVIDIA, respectively.

Metrics We choose `tokens/step` and `tokens/second` as our main metrics, followed by prior speculative decoding works. The former metric measures the accepted length, indicating the accuracy of speculators, while the latter metric reports the overall system throughput. We report top-k accuracy of each head in ablation study to gain more intuitive insight into diverse model architecture.

4.2 End-to-end Results

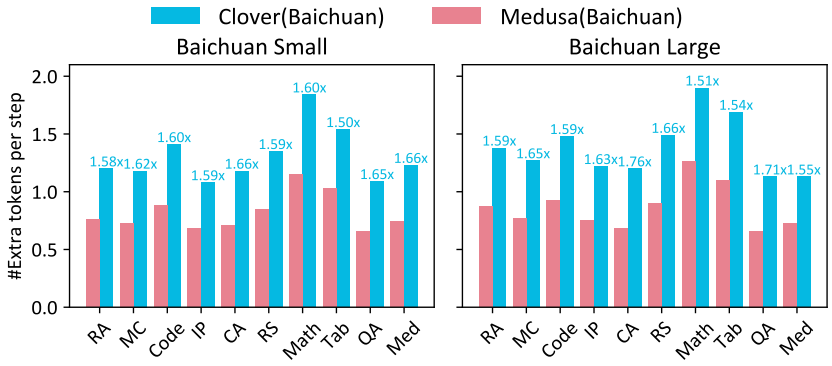


Figure 6: Number of extra generated tokens (excluding the first one) per step on various tasks.

We evaluate the end-to-end performance at different batch sizes. As mentioned in Section 1, in real-time serving environment, system need to compute requests with large batch sizes and easily meet the computational bound. Thus we set token tree size to 4 for both speculative decoding methods. We also investigate and find that further expansion of the tree sampling size leads to marginal effects (see Appendix A.2).

Figure 6 illustrates the average number of tokens generated per step for `Clover` and Medusa methods on different tasks. Note that the value on the vertical axis is the **extra** tokens per step, excluding the actual token generated by target model, which more accurately reflects the performance of the speculator. `Clover` generates 50% - 76% more extra tokens per step than Medusa method on all tasks, highlighting its superiority over Medusa architecture in terms of speculator accuracy.

[1]Cosine decay is applied to the learning rate.

Model Size	Task	Approach	Tokens/second and Improvement over Vanilla Decoding					
			bs=4	bs=8	bs=16	bs=24	bs=32	bs=48
Small	**CA**	`Clover` (Baichuan)	**195.9** **+44%**	**373.1** **+40%**	**615.5** **+22%**	**872.0** **+19%**	**1035.2** **+14%**	1352.3 -39%
		Medusa (Baichuan)	160.3 +18%	343.6 +28%	554.3 +10%	778.7 +6%	948.2 +4%	1246.0 −43%
		Vanilla	135.5	266.4	501.3	731.6	903.2	**2217.4**
	Math	`Clover` (Baichuan)	**232.2** **+91%**	**411.3** **+76%**	**673.3** **+57%**	**988.4** **+67%**	1137.7 -5%	1462.8 -21%
		Medusa (Baichuan)	187.6 +54%	342.4 +46%	645.2 +50%	786.8 +32%	974.4 −18%	1333.2 −28%
		Vanilla	121.5	233.5	428.5	591.6	**1202.4**	**1874.0**
Large	**CA**	`Clover` (Baichuan)	**169.3** **+86%**	**290.2** **+55%**	**488.5** **+34%**	**638.4** **+23%**	**754.4** **+22%**	**938.2** **+5%**
		Medusa (Baichuan)	132.3 +45%	222.1 +19%	373.3 +2%	486.8 −5%	581.3 −5%	638.0 −28%
		Vanilla	90.7	186.2	362.8	515.5	615.8	887.8
	Math	`Clover` (Baichuan)	**207.3** **+146%**	**342.1** **+103%**	**549.5** **+81%**	**715.4** **+62%**	**874.3** **+63%**	**1067.8** **+36%**
		Medusa (Baichuan)	159.1 +89%	269.8 +60%	401.3 +32%	557.0 +26%	705.7 +31%	781.8 +0%
		Vanilla	84.2	168.3	302.9	440.9	535.2	780.9

Table 1: End-to-end throughput on `Baichuan Small` and `Baichuan Large` with different decoding methods on two tasks, where `bs` in the head means batch size, and Vanilla refers to auto-regressive decoding. Results for other tasks are shown in Appendex A.1.

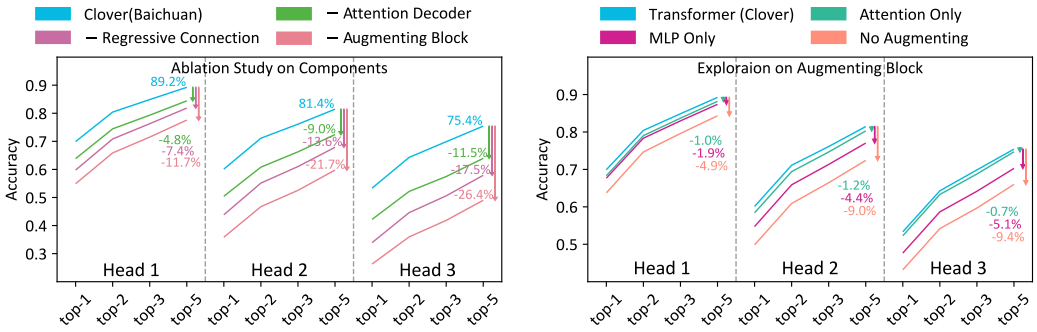
The end-to-end throughput (i.e. `tokens/second`) results are shown in Table 1. Both `Clover` and Medusa speculative decoding methods outperform auto-regressive decoding (at most $2.05\times$ - $2.56\times$ in terms of throughput on `Baichuan Large` model) due to effective hardware utilisation in most scenarios. We also find that the advantage of both speculation decoding methods generally diminishes with increasing batch size. This is because speculative decoding with larger batch size is getting closer to the computational bounded. The performance fluctuation is due to unpredictable random factors during the inference and a not fully optimized implementation of our engine. More results for other tasks can be found in Appendix A.1, `Clover` (Baichuan) still retains its best performance over Medusa(Baichuan) and auto-regressive decoding in all categories.

Moreover, `Clover` decoding generates more tokens per step and achieves higher throughput than Medusa decoding in all scenarios (at most $1.26\times$ and $1.47\times$ for `Baichuan Small` and `Baichuan Large`, respectively[2]), because the gain in head accuracy from the additional components proposed in Clover outweighs their computational overhead. The advantages of our system over Medusa are even more pronounced for larger model sizes, as the speculator module makes up a smaller proportion of the overall model. The sequential knowledge from pre-generated speculation tokens helps the current head to predict next speculation token more accurately, especially when speculating a long multi-token phrase that appears first in the first head, but not at the next-0 token (i.e. the actual output token).

4.3 Ablation Study

In Ablation study, we use top-k accuracy of each head as the metrics to intuitively understand how each component affects accuracy.

[2]The value is the ratio of `Clover` to Medusa throughput.



(a) Ablation Study on Components

(b) Exploration on Augmenting Block

Figure 7: Ablation study on `Baichuan Small` model.

Ablations on Components We start from complete `Clover` (Baichuan) and gradually remove: Attention Decoder, Regressive Connection and Augmenting Block. Note that after removing all the three components, the model architecture becomes identical to the Medusa(Baichuan). Figure 7a shows the accuracy of `Clover` (Baichuan) with difference components enabled.

By removing Attention Decoder, taking the MLP layer as the regressive block instead, the top-5 accuracy of the three heads reduces by $(4.8\%, 9.0\%, 11.5\%)$. This is because MLP layer itself will not distinguish different embedding vectors, make it hard to learn valid sequential knowledge from the entire sentence. If we further disable the Regressive Connection, the accuracy decreases as well (e.g. $2.6\%, 4.6\%, 6.0\%$ top-5 further accuracy loss from the three heads). The removal of regressive connection indicates the removal of sequential dependence knowledge of pre-generated tokens, resulting in accuracy loss. Finally, `Clover` (Baichuan) becomes Medusa(Baichuan) after further disabling Augmenting Block, losing additional $(4.3\%, 8.1\%, 8.9\%)$ top-5 accuracy from all heads, respectively. The missing of augmenting sequential knowledge makes it harder for succeeding to perform speculation.

Overall comparing `Clover` (Baichuan) with Medusa(Baichuan), `Clover` approach brings sequential knowledge from pre-generated speculative tokens as well as the input sentence, improving performance of all speculative heads, especially the latter two. The same observation is also confirmed in Figure 7b.

Exploration on Augmenting Block We further explore the potential variants of Augmenting Block, including: a whole transformer block (the actual architecture in `Clover` (Baichuan)), attention block only , MLP block only and no Augmenting Block. Figure 7b shows accuracy of `Clover` with different types of Augmenting Block equipped. The transformer block contributes $(4.9\%, 9.0\%, 9.4\%)$ top-5 accuracy to the heads compared with no augmenting block, in which the attention block plays the major role (increasing $1.9\%, 4.4\%, 5.1\%$ top-5 accuracy when only attention block enabled for Augmenting Block). While the MLP block only provides $(1.0\%, 1.2\%, 0.7\%)$ accuracy improvement, thus we leave it as an optional component.

The attention mechanism focuses on extracting relationships between tokens in the sentence, making it easier to learn for feature augmentation. Although the performance gain from incorporating MLP is not significant, we chose to enable it in order to improve prediction accuracy, as it has minimal time and memory overhead.

5 Related Works

Speculative Inference Since Speculative decoding for LLM first proposed in [13, 6], multiple optimization technologies has been studied. Tree attention was explored in [16] and widely applied for verifying multiple speculations in a single step. Several early works [12, 15, 17, 19, 28, 27, 11, 7] studied how to improve separated draft models, and some works [22, 10, 9] also explored training-free

draft model architecture, while more recent works such as [5, 3, 23, 8] also drew more attention to the integrated draft model. `Clover` is one of the extension based on such lightweight speculator.

Regressive Speculator There are some recent approaches that also explore the potential superiority of the regressive speculator. Zhang et al. [26] use an MLP layer as a regressive block, and Hydra [2] also introduces an additional block in their implementation. Eagle [14] also introduces a regressive transformer block to speculate. Chimera [25] proposed Trigram Encoder and Full Context Encoder as regressive speculators. The primary distinction of `Clover` is the use of cross Attention Decoder and the exploration on Augmenting Block, with the aim of optimising the utilisation of sequential knowledge derived from both pre-specified tokens and the input sentence. In addition, `Clover` focuses on throughput improvement at larger batch sizes and smaller tree sizes, which has not been sufficiently addressed in previous speculative decoding work.

6 Conclusion

We present `Clover`, an extension of the Medusa method that considers sequential knowledge in speculation generation. `Clover` exploits sequential knowledge from pre-generated speculative tokens (Section 3.1), the entire input sentence (Section 3.3) and their combination (Section 3.2), achieving 11.7% - 26.4% more top-5 accuracy for speculative heads and a $1.26\times$ - $1.47\times$ throughput improvement when deploying `Clover` on `Baichuan Large` model compared with Medusa method (with 50% - 76% more speculative tokens accepted), and at most $2.56\times$ with vanilla auto-regressive decoding. The main contribution to accuracy comes from the latter heads ($+21.7\%$ - $+26.4\%$) compared to $+11.7\%$ for the first head. Such evidence support the view that the auto-regressive mechanism is an effective approach to improve the accuracy of speculation.

References

[1] ChatGPT: Optimizing Language Models for Dialogue, 2022. `https://openai.com/blog/chatgpt/`.

[2] Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. Hydra: Sequentially-dependent draft heads for medusa decoding, 2024.

[3] Nikhil Bhendawade, Irina Belousova, Qichen Fu, Henry Mason, Mohammad Rastegari, and Mahyar Najibi. Speculative streaming: Fast llm inference without auxiliary models, 2024.

[4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, pages 1877–1901, 2020.

[5] Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads, 2024.

[6] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling, 2023.

[7] Ziyi Chen, Xiaocong Yang, Jiacheng Lin, Chenkai Sun, Kevin Chen-Chuan Chang, and Jie Huang. Cascade speculative drafting for even faster llm inference, 2024.

[8] Cunxiao Du, Jing Jiang, Xu Yuanchen, Jiawei Wu, Sicheng Yu, Yongqi Li, Shenggui Li, Kai Xu, Liqiang Nie, Zhaopeng Tu, and Yang You. Glide with a cape: A low-hassle method to accelerate speculative decoding, 2024.

[9] Yichao Fu, Peter Bailis, Ion Stoica, and Hao Zhang. Break the sequential dependency of llm inference using lookahead decoding, 2024.

[10] Zhenyu He, Zexuan Zhong, Tianle Cai, Jason D. Lee, and Di He. Rest: Retrieval-based speculative decoding, 2024.

[11] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Hasan Genc, Kurt Keutzer, Amir Gholami, and Sophia Shao. Speed: Speculative pipelined execution for efficient decoding, 2024.

[12] Sehoon Kim, Karttikeya Mangalam, Suhong Moon, Jitendra Malik, Michael W Mahoney, Amir Gholami, and Kurt Keutzer. Speculative decoding with big little decoder. In A. Oh, T. Neumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, editors, *Advances in Neural Information Processing Systems*, volume 36, pages 39236–39256. Curran Associates, Inc., 2023.

[13] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286. PMLR, 23–29 Jul 2023.

[14] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty, 2024.

[15] Xiaoxuan Liu, Lanxiang Hu, Peter Bailis, Ion Stoica, Zhijie Deng, Alvin Cheung, and Hao Zhang. Online speculative decoding, 2023.

[16] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ASPLOS '24, page 932–949, New York, NY, USA, 2024. Association for Computing Machinery.

[17] Giovanni Monea, Armand Joulin, and Edouard Grave. Pass: Parallel speculative sampling, 2023.

[18] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019.

[19] Benjamin Spector and Chris Re. Accelerating llm inference with staged speculative decoding, 2023.

[20] Daliang Xu, Wangsong Yin, Xin Jin, Ying Zhang, Shiyun Wei, Mengwei Xu, and Xuanzhe Liu. Llmcad: Fast and scalable on-device large language model inference, 2023.

[21] Aiyuan Yang, Bin Xiao, Bingning Wang, Borong Zhang, Ce Bian, Chao Yin, Chenxu Lv, Da Pan, Dian Wang, Dong Yan, Fan Yang, Fei Deng, Feng Wang, Feng Liu, Guangwei Ai, Guosheng Dong, Haizhou Zhao, Hang Xu, Haoze Sun, Hongda Zhang, Hui Liu, Jiaming Ji, Jian Xie, JunTao Dai, Kun Fang, Lei Su, Liang Song, Lifeng Liu, Liyun Ru, Luyao Ma, Mang Wang, Mickel Liu, MingAn Lin, Nuolan Nie, Peidong Guo, Ruiyang Sun, Tao Zhang, Tianpeng Li, Tianyu Li, Wei Cheng, Weipeng Chen, Xiangrong Zeng, Xiaochuan Wang, Xiaoxi Chen, Xin Men, Xin Yu, Xuehai Pan, Yanjun Shen, Yiding Wang, Yiyu Li, Youxin Jiang, Yuchen Gao, Yupeng Zhang, Zenan Zhou, and Zhiying Wu. Baichuan 2: Open large-scale language models, 2023.

[22] Nan Yang, Tao Ge, Liang Wang, Binxing Jiao, Daxin Jiang, Linjun Yang, Rangan Majumder, and Furu Wei. Inference with reference: Lossless acceleration of large language models, 2023.

[23] Hanling Yi, Feng Lin, Hongbin Li, Peiyang Ning, Xiaotian Yu, and Rong Xiao. Generation meets verification: Accelerating large language model inference with smart parallel auto-correct decoding, 2024.

[24] Boxiang Yun, Yan Wang, Jieneng Chen, Huiyu Wang, Wei Shen, and Qingli Li. Spectr: Spectral transformer for hyperspectral pathology image segmentation, 2021.

[25] Ziqian Zeng, Jiahong Yu, Qianshi Pang, Zihao Wang, Huiping Zhuang, Hongen Shao, and Xiaofeng Zou. Chimera: A lossless decoding method for accelerating large language models inference by fusing all tokens, 2024.

[26] Aonan Zhang, Chong Wang, Yi Wang, Xuanyu Zhang, and Yunfei Cheng. Recurrent drafter for fast speculative decoding in large language models, 2024.

[27] Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft & verify: Lossless large language model acceleration via self-speculative decoding, 2023.

[28] Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. Distillspec: Improving speculative decoding via knowledge distillation, 2024.

A Appendix

A.1 More End-to-end Results on Baichuan Large model

Task	Approach	Tokens/second					
		bs=4	bs=8	bs=16	bs=24	bs=32	bs=48
RA	`Clover`	**120.7**	**186.2**	**270.2**	**309.5**	**346.2**	**372.5**
	Medusa	108.0	160.7	234.5	271.4	287.2	300.3
	Vanilla	67.9	117.5	195.9	237.6	279.4	295.1
MC	`Clover`	**121.1**	**175.9**	**262.7**	**324.4**	**383.8**	**408.9**
	Medusa	101.9	152.9	222.4	282.7	303.5	324.4
	Vanilla	72.1	127.4	204.4	266.9	324.6	346.2
Code	`Clover`	**165.6**	**266.0**	**411.5**	**506.8**	**622.3**	**717.6**
	Medusa	130.2	218.7	354.1	442.6	515.5	562.3
	Vanilla	81.4	152.5	274.6	376.3	447.5	539.0
IP	`Clover`	**145.6**	**240.5**	**361.9**	**467.8**	**542.8**	**650.9**
	Medusa	116.4	196.9	299.9	389.4	470.0	501.3
	Vanilla	74.9	145.5	268.9	348.3	416.6	531.2
CA	`Clover`	**169.3**	**290.2**	**488.5**	**638.4**	**754.4**	**938.2**
	Medusa	132.3	222.1	373.3	486.8	581.3	638.0
	Vanilla	90.7	186.2	362.8	515.5	615.8	887.8
RS	`Clover`	**194.3**	**330.0**	**526.1**	**733.3**	**837.8**	**1076.1**
	Medusa	154.4	248.9	423.3	552.3	688.6	804.9
	Vanilla	101.6	188.2	374.4	541.9	701.8	984.1
Math	`Clover`	**207.3**	**342.1**	**549.5**	**715.4**	**874.3**	**1067.8**
	Medusa	159.1	269.8	401.3	557.0	705.7	781.8
	Vanilla	84.2	168.3	302.9	440.9	535.2	780.9
Tab	`Clover`	**178.8**	**271.6**	**433.9**	**553.1**	**659.1**	**884.3**
	Medusa	123.3	186.5	360.3	417.2	587.9	648.0
	Vanilla	66.7	143.0	257.3	365.3	413.0	654.4
QA	`Clover`	**164.9**	**271.1**	**447.2**	**573.0**	**710.0**	802.3
	Medusa	127.5	213.4	349.2	457.3	565.4	623.6
	Vanilla	88.6	179.9	343.2	498.2	617.2	**805.4**
Med	`Clover`	**167.7**	**282.3**	**472.4**	**609.4**	**765.9**	**894.5**
	Medusa	139.2	229.0	382.7	501.5	611.7	690.0
	Vanilla	90.3	186.4	361.1	518.3	628.9	891.0

Table 2: End-to-end throughput on `Baichuan Large` model for different tasks.

A.2 Token Tree Size

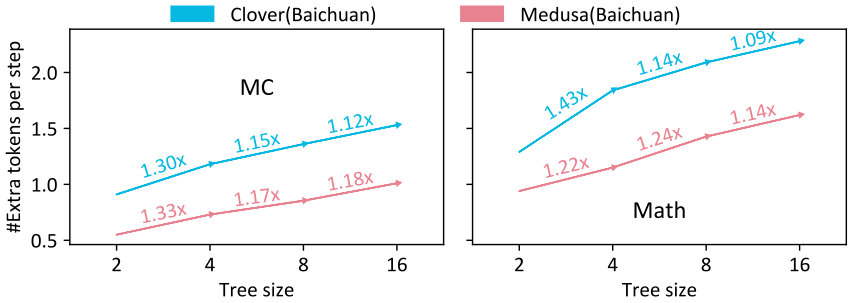


Figure 8: Extra tokens per step v.s. token tree size on `Baichuan Small` model with multi-turn conversation (**MC**) task and math(**Math**) task.

We find marginal effects in token tree size. As the token tree size grows larger, the acceptance length is still increases but at a slower rate. Note that the horizontal axis in Figure 8 is in exponential scale, while the vertical axis is linear. Since doubling the token tree size brings much more computation and gets closer to the computational bound, we set token tree size to 4 in our evaluation to adapt to the large batch size scenario.